Chapter 2: Understanding What You Are Testing - LLMs and AI Agents

Series: From Manual Tester to AI QA Practitioner: A Structured Six-Week Tutorial Series on Testing LLMs and AI Agents

Author: Tanvi Mittal

Series DOI: https://doi.org/10.5281/zenodo.18972360

Chapter: 2 of 6 **Week:** 2 **Estimated Time:** 4–5 hours

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

How to Cite This Chapter

Mittal, T. (2026). *Chapter 2: Understanding What You Are Testing LLMs and AI Agents*. In From Manual Tester to AI QA Practitioner: A Structured Six-Week Tutorial Series on Testing LLMs and AI Agents (v[1]). Zenodo. https://doi.org/10.5281/zenodo.18972360

Learning Outcomes

By the end of this chapter, you will be able to:

- Describe how an LLM generates text at the level of detail needed to write meaningful test cases
- Distinguish between a standalone LLM and an AI agent
- Map the components of an AI agent to their testable surfaces
- Explain what a system prompt is and why it is a primary test target
- Complete a System Under Test (SUT) card for a real AI product

What kinds of failure it produces and understanding that mechanism at a conceptual level will make you a significantly more effective tester.

---

[1] . Why Testers Need to Understand the System

In traditional QA, you do not need to know how a database engine works to test that a form saves user data correctly. The internal implementation is the developer's concern. Your concern is the observable behavior at the boundary of the system.

AI systems change this somewhat. Not because you need to understand the mathematics of neural networks you do not. But because the way an LLM generates output directly shapes

Think of it this way: a tester who understands that LLMs predict the next most likely word rather than retrieving a correct answer from a database will naturally ask different questions than one who does not. They will probe for confident wrongness rather than just wrong answers.

They will test what happens when the model has no good information to work with. They will notice when a response sounds right but has no grounding in fact.

That conceptual understanding is what this chapter builds. And by the end of it, you will also understand the distinction between testing an LLM and testing an AI agent which is a different and more complex challenge.

---

2. How an LLM Works - What a Tester Needs to Know

2.1 Prediction, Not Retrieval

The single most important thing to understand about an LLM is this: it does not look up answers. It predicts them.

When you type a question into an LLM-powered product, the model does not search for a database, consult a document, or retrieve a verified fact. It generates a response by predicting, token by token, what words are most likely to follow the words already in the conversation based on patterns it learned from an enormous amount of text during training.

A **token** is roughly three-quarters of a word on average. The sentence "The customer has not received their order" is approximately eight tokens. The model processes everything - your prompt, the conversation history, and its own previous output as a sequence of tokens.

This prediction process is why LLMs are fluent and why they hallucinate. Fluency comes from the same mechanism as incorrectness: the model produces what is statistically plausible, not what is verifiably true.

What this means for testing: Any input that sounds like something the model has seen a lot of will produce a confident-sounding response regardless of whether that response is accurate. Your test cases need to probe confidence versus accuracy specifically, not assume that fluent output is correct output.

2.2 Context Windows

Every LLM has a **context window** the total amount of text it can process at one time, including your prompt, any conversation history, any documents it has been given, and its own prior responses.

If a conversation exceeds the context window, the model loses access to the earliest parts of the exchange. It cannot refer to what was said at the beginning of a long conversation because that content has effectively been forgotten.

What this means for testing: Long conversations are a distinct test category. A behavior that holds for the first ten exchanges may break down after fifty because the model's earlier instructions have dropped out of its context window. Testing for behavioral consistency over extended interactions is a specific AI QA discipline.

2.3 Temperature and Variation

Most LLM deployments have a setting called **temperature** that controls how much variation appears in the model's outputs. A low temperature setting makes the model more consistent and predictable, it will tend to produce very similar outputs for the same input. A high temperature setting makes it more varied and creative the same input may produce quite different outputs each time.

You as a tester will not usually control this setting directly, it is set by the product developer. But you need to know it exists, because it explains why the same LLM may behave very consistently in one product and very variably in another.

What does this means for testing: When you observe high output variation in your testing, it may be a product configuration decision, not a model failure. Part of your role is understanding whether the observed variation is within the product team's intended range.

2.4 Instruction Tuning and Guardrails

A base LLM, trained only on raw text will generate text that continues whatever pattern it has been given, including harmful, offensive, or dangerous content. The models you encounter in consumer and enterprise products have been further trained using a process called **instruction tuning** and **reinforcement learning from human feedback (RLHF)**, which teaches the model to follow instructions, decline certain requests, and behave according to defined safety policies.

On top of that, developers add **guardrails** additional layers of filtering or instruction that prevent the model from producing specific types of output, or that redirect certain inputs.

What does this means for testing: Guardrails and instruction tuning are not perfect. They are the primary targets of adversarial testing, which you will practice in Chapter 4. For now, the key insight is that the safety behavior you observe in a product is a combination of the base model's training and the developer's added controls and both can fail.

---

3. The System Prompt - Your Most Important Test Target

3.1 What It Is

When a developer builds a product on top of an LLM, they almost always give the model a set of instructions that define its role, its behavior, its restrictions, and its context. These instructions are called the **system prompt** (also referred to as the system message, meta-prompt, or instruction context depending on the platform).

The system prompt is invisible to the end user in most products. You interact with the AI assistant, but you do not see the instructions the developer has given it. A system prompt for a customer support chatbot might say something like:

> *"You are a helpful support agent for Acme Corporation. You only answer questions about Acme products. If asked about competitors, politely decline. Never discuss pricing direct the customer to the sales team. Always maintain a professional and empathetic tone."*

Every behavior defined in that system prompt is a **testable claim**. The developer has made assertions about how the system will behave and your job is to verify whether the system actually behaves that way.

3.2 Why It Matters for Testing

The system prompt is the closest thing an AI product has to a functional specification. It tells you:

- What the system is supposed to do
- What it is supposed to refuse
- What persona or tone should it maintain
- What boundaries have been set

When you do not have access to the system prompt directly which is common your testing process includes **inferring** what the system prompt likely says, based on the product's observable behavior. This is a core AI QA skill and the subject of this chapter's exercise.

3.3 System Prompt Vulnerabilities

Because the system prompt defines the product's behavior, it is also a primary attack surface. Users can attempt to override, extract, or manipulate the system prompt through carefully crafted inputs — a class of attacks covered in detail in Chapter 4. For now, simply understand that the system prompt is both the specification and a vulnerability, and that your test design should account for both dimensions.

---

4. From LLM to AI Agent - A Critical Distinction

4.1 What a Standalone LLM Does

A standalone LLM takes text input and produces text output. That is its complete range of action. It cannot send an email, book a meeting, search the web, execute code, or modify a database. It reads and it writes.

When you interact with a chatbot that is a thin layer on top of an LLM, you are essentially interacting with a standalone LLM. Its output has no direct effect on the world beyond the conversation.

4.2 What an AI Agent Does

An **AI agent** is a system where an LLM is connected to tools, memory, and the ability to take actions and then given a goal and allowed to determine how to pursue it.

A simple example: an AI agent for expense management might be given the goal "process all pending expense reports submitted this week." To pursue that goal, it might:

1. Query a database to retrieve pending reports

2. Read each report and evaluate it against policy
3. Approve or flag reports based on that evaluation
4. Send notification emails to submitters
5. Update the expense management system with its decisions

Each of those steps involves the LLM directing a tool call and the output of each step feeds into the next decision. The agent is not just producing text. It is taking actions with real-world consequences.

4.3 The Four Components of an AI Agent

Understanding the structure of an AI agent helps you identify what to test. Most AI agents consist of four components:

Component	What It Does	What Can Go Wrong
Model	The LLM that reasons, plans, and generates outputs	Hallucination, boundary violations, instruction drift
Memory	Storage of information across steps or sessions, short-term (within a task) and long-term (across tasks)	Memory corruption, outdated context, privacy leaks
Tools	External capabilities the agent can use web search, APIs, code execution, file access, email	Tool misuse, excessive permissions, incorrect tool selection
Orchestration	The logic that sequences the agent's steps and manages its goal pursuit	Infinite loops, goal drift, failure to stop, compounding errors

Each of these components is a test surface. Failures in any one of them can produce quality or safety issues and failures can compound across components in ways that are difficult to predict.

4.4 Why Agent Testing Is More Complex Than LLM Testing

When you test a standalone LLM, the scope of a failure is bounded by the conversation. A bad response is a bad response the user can correct it, ignore it, or start over.

When you test an AI agent, failures can have downstream consequences. An agent that misclassifies an expense report may send incorrect notifications, update a system with wrong data, and block a legitimate reimbursement all as the result of a single reasoning error. And because agents act sequentially, early errors propagate through subsequent steps, compounding before anyone can intervene.

This is why Chapters [2] in this series is dedicated entirely to agent testing. For now, your goal is to hold the distinction clearly: **LLM testing is about evaluating outputs. Agent testing is about evaluating actions and their consequences.**

Attack Surface	Description	Examples
User prompt	The text the end user sends to the system	Questions, commands, multi-turn conversation
System prompt	The developer instructions that define the model's behavior	Can be targeted for extraction or override
Retrieved content	Documents, web pages, or database records the LLM processes	Can contain embedded malicious instructions
Tool inputs/outputs	Data flowing into and out of agent tools	Can be manipulated to cause unintended tool behavior
Memory stores	Information persisted between agent sessions	Can be poisoned with false context
Model parameters	Configuration settings like temperature and context limits	Misconfigurations can expand the attack surface

You do not need to test every attack surface in your first test campaign. But you need to know they exist, so that your test design does not assume the only input path is the user-facing chat interface.

---

6. AI Testing Vocabulary Extended Reference

The following terms build on the vocabulary introduced in Chapter 1. You will encounter all of them in the exercises and chapters ahead.

Term	Plain Language Definition
System prompt	Developer-written instructions given to the LLM before user interaction begins

---

[2] . The Attack Surface of an AI System

The term **attack surface** comes from security testing. It refers to all the points where a system can be accessed, interacted with, or manipulated by a user including malicious users.

For a traditional web application, the attack surface includes input fields, API endpoints, authentication mechanisms, and file upload functions. For an AI system, the attack surface is different and broader.

User prompt	The input the end user sends to the system
Multi-turn conversation	A conversation consisting of multiple exchanges, as opposed to a single question and answer
Tool call	When an AI agent instructs an external tool (API, search engine, code executor) to perform an action
Agentic system	Any AI system where an LLM directs actions, not just generates text
RAG	Retrieval-Augmented Generation a technique where the LLM is given retrieved documents to ground its responses
Memory (shortterm)	Context the agent retains within a single task or session
Memory (longterm)	Context the agent retains across multiple sessions
Goal drift	When an agent deviates from its intended objective during a multi-step task
Prompt leakage	When a model reveals the contents of its system prompt to a user who should not have access to it
Refusal	When a model declines to respond to a request, typically due to a safety guardrail
Jailbreak	An input designed to bypass a model's safety guardrails
Over-permission	When an AI agent has access to tools or data beyond what its task requires

7. Hands-On Exercise: System Under Test (SUT) Card

What You Need

- Access to any publicly available AI-powered product a chatbot on a company website, a coding assistant, a writing assistant, or a customer-facing AI tool
- The SUT card template below
- 60–90 minutes

What You Are Going to Do

In traditional testing, before you write a single test case, you document what you are testing the System Under Test (SUT). You describe its purpose, its inputs, its outputs, and its known constraints.

AI products rarely come with complete specifications. Your job in this exercise is to **infer the SUT documentation** through structured interaction and then use that documentation to identify three things you would want to verify as a tester.

This is a real AI QA skill. In many organizations, AI QA practitioners receive a product brief or a demo, not a specification. The ability to derive a testable understanding from observation is fundamental.

Step 1: Choose Your Subject

Select one of the following types of AI product to use for this exercise. Use a free, publicly accessible version:

- Customer support chatbot on any company's website (retail, software, or services)
- ChatGPT, Claude, or Gemini interact with it as if it were a writing assistant with a defined scope you set yourself
- A coding assistant (GitHub Copilot free tier, Google's code assist, or similar)
- Any AI agent demo listed in /exercises/chapter-02/agent-demo-list.md

Step 2: Interact and Observe

Spend 20–30 minutes interacting with the product. Your goal is not to use it, it is to observe it. As you interact, try to answer the following questions through observation alone:

1. What is this product supposed to do? What is its primary purpose?
2. What types of input does it accept? (Text only? Files? Voice?)
3. What does it consistently refuse or redirect?
4. What persona or tone does it maintain?
5. Is there evidence it has access to external tools (web search, database lookup, calendar, etc.)?
6. Does it appear to have memory of earlier parts of your conversation?

Step 3: Complete the SUT Card

Fill in the following template based on your observations:

System Under Test (SUT) Card

SUT ID: CH02-EX01-[your initials]-[date]
Product Name / URL: [Name or URL of the AI product you observed]
Access Method: [Free tier / Demo / Public chatbot]
Date of Observation: [Date]

Inferred Purpose:
[One to two sentences: what does this system appear to be designed to do?]

Input Types Observed:
[List all input types you observed the system accepting]

Output Types Observed:
[Text? Structured lists? Code? Images? Actions? List what you saw]

Inferred System Prompt Elements:
[List the behavioral rules or constraints you observed that suggest system prompt instructions. Format each as: "The system appears to be instructed to [X] because [evidence]."]

Evidence of Tool Use:
[Did the system appear to use any external tools? Describe what you observed and how you inferred it.]

Evidence of Memory:
[Did the system appear to remember context from earlier in your conversation? Describe your observation.]

Observed Refusals:
[List any inputs the system declined to respond to. Record the input and the system's response.]

Three Test Targets:
Based on your observations, list three specific behaviors you would want to verify as a tester:
1.
2.
3.

Unknowns and Gaps:
[List anything you could not determine from observation alone things you would need access to the system prompt or developer documentation to know.]

Step 4: Reflect

Write two to three sentences answering this question:

> *What is the most important thing you still do not know about this system and what would be the consequence of designing test cases without that knowledge?*

What to Submit / Save

Save your completed SUT card. You will reference it in Chapter 5 when you design agent-level test cases, and it is a concrete artifact demonstrating structured analytical thinking in AI QA.

---

8. Key Takeaway

You cannot write good test cases for a system you do not understand. For AI systems, understanding the system means understanding the model's instructions, its access to tools, and the boundaries its developers intended even when those boundaries are not documented.

The SUT card is how you make that understanding explicit, testable, and shareable with your team. It is the AI QA equivalent of a requirements review and it is something most teams are not yet doing systematically. Practitioners who do it well stand out.

Chapter Summary

Concept	One-Line Summary
LLM prediction	LLMs predict likely output they do not retrieve verified answers
Context window	LLMs can only process a fixed amount of text at a time long conversations can cause earlier context to be lost
System prompt	Developer instructions that define behavior your primary specification and a key test target
AI agent	An LLM connected to tools, memory, and action a fundamentally more complex test subject
Four agent components	Model, memory, tools, orchestration each is a distinct test surface
Attack surface	the ways a user or adversary can interact with or manipulate an AI system

What Is Coming in Chapter 3

In Chapter 3, you move from observation to execution. You will write your first structured AI test suite using a behavior-based test case format adapted for LLM outputs and you will run it against a live LLM using a free playground environment. You will also be introduced to evaluation rubrics: the mechanism that makes subjective AI output assessment consistent and repeatable.

References and Further Reading

- Weng, L. (2023). *LLM Powered Autonomous Agents*. Lilian Weng's Blog. https://lilianweng.github.io/posts/2023-06-23-agent/
- Wei, J., et al. (2022). Emergent abilities of large language models. *Transactions on Machine Learning Research*. https://arxiv.org/abs/2206.07682
- Perez, F., & Ribeiro, I. (2022). *Ignore previous prompt: Attack techniques for language models*. https://arxiv.org/abs/2211.09527
- OWASP. (2025). *OWASP Top 10 for Large Language Model Applications 2025*. https://owasp.org/www-project-top-10-for-large-language-model-applications/
- Anthropic. (2024). *Building effective agents*. https://www.anthropic.com/research/buildingeffective-agents
- Bommasani, R., et al. (2021). On the opportunities and risks of foundation models. *Stanford CRFM*. https://arxiv.org/abs/2108.07258

This chapter is part of the series: From Manual Tester to AI QA Practitioner (Zenodo DOI: https://doi.org/10.5281/zenodo.18972360). Published under CC BY 4.0. Please cite using the citation block at the top of this document.